



Dynamic Programming

Optimization Through Memorization

Java 8+

THIRAVUGAL



Session Structure

1. Why Dynamic Programming?
2. Recursion vs Memoization
3. Tabulation (Bottom-Up)
4. Classic DP Problems
5. 1D & 2D DP patterns
6. Optimization strategies
7. Interview problem solving



What is Dynamic Programming?

Dynamic Programming (DP):

Break problem into overlapping subproblems

Store solutions

Reuse them

Avoid recomputation.



When to Use DP?

Problem has:

1. Optimal substructure
2. Overlapping subproblems

If both exist → DP candidate.

DP vs Recursion vs Greedy

Approach	Recompute?	Optimal?
Recursion	Yes	Maybe
Greedy	No	Not always
DP	No	Yes



Fibonacci — Naive

```
static int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

Time: $O(2^n)$



Fibonacci — Memoization

```
static int fibMemo(int n, int[] dp) {  
    if (n <= 1) return n;  
    if (dp[n] != -1) return dp[n];  
  
    dp[n] = fibMemo(n - 1, dp) +  
            fibMemo(n - 2, dp);  
  
    return dp[n];  
}
```

Time: $O(n)$

Space: $O(n)$

Fibonacci — Tabulation

```
static int fibTab(int n) {  
    if (n <= 1) return n;  
  
    int[] dp = new int[n + 1];  
    dp[0] = 0;  
    dp[1] = 1;  
  
    for (int i = 2; i <= n; i++)  
        dp[i] = dp[i - 1] + dp[i - 2];  
  
    return dp[n];  
}
```

Time: $O(n)$

Space: $O(n)$

Space Optimization

```
static int fibOptimized(int n) {  
    if (n <= 1) return n;  
  
    int prev2 = 0, prev1 = 1;  
  
    for (int i = 2; i <= n; i++) {  
        int curr = prev1 + prev2;  
        prev2 = prev1;  
        prev1 = curr;  
    }  
  
    return prev1;  
}
```

Space: $O(1)$



0/1 Knapsack

Choose items to maximize value without exceeding capacity.

Time: $O(n * W)$

THIRAVUGAL

0/1 Knapsack Code

```
static int knapsack(int[] weight, int[] value, int W) {  
    int n = weight.length;  
    int[][] dp = new int[n + 1][W + 1];  
  
    for (int i = 1; i <= n; i++) {  
        for (int w = 0; w <= W; w++) {  
            if (weight[i - 1] <= w)  
                dp[i][w] = Math.max(  
                    value[i - 1] +  
                    dp[i - 1][w - weight[i - 1]],  
                    dp[i - 1][w]  
                );  
            else  
                dp[i][w] = dp[i - 1][w];  
        }  
    }  
  
    return dp[n][W];  
}
```

Longest Common Subsequence (LCS)

Time: $O(n * m)$

```
static int lcs(String s1, String s2) {  
    int n = s1.length();  
    int m = s2.length();  
    int[][] dp = new int[n + 1][m + 1];  
  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= m; j++) {  
            if (s1.charAt(i - 1) ==  
                s2.charAt(j - 1))  
                dp[i][j] =  
                    1 + dp[i - 1][j - 1];  
            else  
                dp[i][j] =  
                    Math.max(dp[i - 1][j],  
                             dp[i][j - 1]);  
        }  
    }  
  
    return dp[n][m];  
}
```



Longest Increasing Subsequence (LIS)

Naive DP: $O(n^2)$



Coin Change (Min Coins)

Time: $O(n * \text{amount})$

```
static int coinChange(int[] coins, int amount) {  
    int[] dp = new int[amount + 1];  
    java.util.Arrays.fill(dp, amount + 1);  
    dp[0] = 0;  
  
    for (int i = 1; i <= amount; i++) {  
        for (int coin : coins) {  
            if (coin <= i)  
                dp[i] = Math.min(dp[i],  
                                1 + dp[i - coin]);  
        }  
    }  
  
    return dp[amount] > amount ? -1 : dp[amount];  
}
```



Grid Path Problem

Count ways to reach bottom-right.

Time: $O(n * m)$

THIRAVUGAL

Grid Path Code

```
static int uniquePaths(int m, int n) {  
    int[][] dp = new int[m][n];  
  
    for (int i = 0; i < m; i++)  
        dp[i][0] = 1;  
  
    for (int j = 0; j < n; j++)  
        dp[0][j] = 1;  
  
    for (int i = 1; i < m; i++)  
        for (int j = 1; j < n; j++)  
            dp[i][j] =  
                dp[i - 1][j] +  
                dp[i][j - 1];  
  
    return dp[m - 1][n - 1];  
}
```




Real World Use Case — Finance

Portfolio optimization.

DP finds best profit combination.



Real World Use Case — Text Processing

Spell correction:

Minimum edit distance (DP).



DP Design Steps

1. Define state
2. Define recurrence
3. Base cases
4. Choose memoization or tabulation
5. Optimize space

Complexity Summary

Problem	Time
Fibonacci	$O(n)$
Knapsack	$O(nW)$
LCS	$O(nm)$
Coin Change	$O(n * \text{amount})$



Common Mistakes

- Wrong state definition
- Incorrect recurrence
- Missing base case
- Not initializing DP table properly
- Forgetting space optimization



Interview Strategy

Ask:

1. Is problem exponential in recursion?
2. Are subproblems repeating?
3. Can results be stored?
4. What is state representation?



Performance Thought Exercise

Without DP:

Fibonacci(50) → billions of calls.

With DP:

Only 50 calls.

Massive improvement.

Summary

Dynamic Programming:

- Eliminates recomputation
- Converts exponential $\rightarrow$ polynomial
- Based on memoization or tabulation
- Core of advanced algorithm design

Master:

- State design
- Recurrence building
- 2D DP tables
- Space optimization